Memory Management and Garbage Collection Strategies in High-Level Quantum Programming Languages

Rajveer Chopra
Reg. No.: 24BCE0122
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, Tamil Nadu, India
rajveer.singhchopra2024@vitstudent.ac.in

***Abstract*—High-level quantum languages let programmers allocate temporary qubits as freely as local variables. Returning those qubits is a memory-management problem without a classical counterpart: a live qubit cannot be copied, inspected or silently dropped, so neither copying nor tracing garbage collection applies. This paper presents QRECLAIM, a compile-time garbage collector for qubits built on a linear intermediate representation (QR-IR). A forward basis-state abstract interpretation computes, for every ancilla, the reversible trace that produced it, its dependencies and an uncomputation deadline. An uncomputation-aware extension of live-variable analysis then places each uncomputation at the earliest provably safe point. The same framework reclaims measured qubits by early measurement and reset, deletes ancillas that are never read, and statically validates that every freed qubit is $|0\rangle$. An earliest-ready-first linear-scan allocator exposes the width–depth trade-off as a single qubit budget. On six benchmark families (29 instances) QRECLAIM reduces the number of physical qubits by 53.8% on average relative to compilation without reuse, and its depth-neutral mode saves 37.7% with no increase in depth. A bug-injection study shows that the static checks detect all 175 faulty mutants at 94.6% precision.**

***Index Terms*—quantum compilers, uncomputation, qubit reuse, garbage collection, data-flow analysis, register allocation, no-cloning theorem**

I. INTRODUCTION

Quantum processors offer tens to a few hundred qubits, so the number of qubits a program needs (its *width*) decides whether it can run at all. At the same time, programs are increasingly written in high-level languages such as Silq [1], Qrisp [2] and Qurts [3], where temporary qubits (*ancillas*) are declared like local variables and returned automatically. Returning them safely and early is the quantum analogue of memory management.

Classical garbage collection does not carry over. A copying collector moves objects, but a quantum state cannot be copied [4]. A tracing collector inspects the heap, but measuring a qubit disturbs it. Above all, a dead temporary is usually *entangled* with live data, and discarding it destroys the interference that quantum algorithms rely on. Such a qubit can be freed only by *uncomputing* it, i.e. running the gates that produced it in reverse [5], which is valid only while its inputs are unchanged. Mid-circuit measurement and reset offer a second route for qubits whose value is no longer needed coherently [6], [7]. A compiler must therefore answer four questions:

1) *Safety:* when can a qubit be reclaimed without aliasing two states onto one qubit or leaving entangled garbage behind?
2) *Placement:* where between an ancilla's last use and the next write to its inputs should its uncomputation go, given that ancillas computed from other ancillas extend each other's lifetimes?
3) *Unification:* can uncomputation-based and measurement-based reclamation be decided by one analysis?
4) *Allocation:* reusing a qubit serialises work that could run in parallel (a false dependency, as in register allocation). How should width be traded against depth?

QRECLAIM answers these with standard compiler machinery adapted to quantum constraints. Its contributions are:

- a **linear IR with static semantics** whose typing rules forbid aliasing and whose T-FREE rule is discharged by a static validator, so a freed qubit is provably $|0\rangle$ (§III-B–III-C);
- a **basis-state abstract interpretation and an uncomputation-aware liveness analysis**, written as data-flow equations with a legality condition and a soundness argument (§III-D);
- a **unified reclamation pass** for uncomputation, early measurement and reset, and dead-garbage elimination, applied hierarchically before and after the lowering of multi-controlled gates (§III-E);
- an **earliest-ready-first (ERF) linear-scan allocator** with an explicit qubit budget and a depth-neutral mode (§III-G);
- an **evaluation** against no reuse, lexical reclamation, a strong baseline that combines prior techniques, and the external framework Qrisp, plus a bug-injection study of the static checks (§V).

II. Literature Review

A. High-level languages and uncomputation

Silq [1] introduced safe automatic uncomputation. Its `qfree` and `const` annotations identify reversible computations and unchanged inputs, temporaries are uncomputed implicitly, and unphysical programs are rejected by the type system. Qurts [3] carries Rust-style affine types with lifetimes over to qubits: inside its lifetime a value may be dropped and is then uncomputed automatically. Qrisp [2] is a Python-embedded framework whose uncomputation follows the Unqomp approach [8]. Its compiler maps qubits of deleted variables onto free physical qubits, and its `workspace` option grants extra qubits to reduce depth. Unqomp [8] synthesises uncomputation on a circuit graph, Reqomp [9] adds a space constraint and trades recomputation for ancillas, and SQUARE [10] decides at each call site whether to uncompute or keep garbage using a cost model that accounts for locality and parallelism. All of these build on Bennett's compute–copy–uncompute construction [5].

B. Qubit reuse by measurement and reordering

CaQR [6] reuses a qubit after its final measurement by inserting a reset, and reports resource reductions of up to 80%. DeCross *et al.* [7] formulate measurement-based reuse exactly as a constraint program and greedily. Jiang [11] recasts circuit-level qubit recycling as matrix triangularisation, proves it NP-hard and gives a dependency-graph heuristic, which justifies heuristic reordering. Closest to our scheduler, Reichental *et al.* [12] search for a topological order of a high-level program that maximises the reuse of uncomputed qubits, with heuristics that trade width for depth.

C. Compiler frameworks and static analysis techniques

Industrial frameworks such as Qiskit [13] compile through a pass manager over a DAG IR. They lower and optimise circuits but treat qubits as fixed registers, so they do not reclaim them. Static analysis of quantum programs is mostly *semantic*: Yu and Palsberg [14] define an abstract interpretation whose abstract states are projections on small qubit subsets, which lets assertions be verified in polynomial time. Our analysis is *structural* instead. It tracks reversible traces, dependencies and lifetimes, which is what reclamation needs, and it transfers classical live-variable analysis and linear-scan register allocation [15] to qubits. Two classical notions do not carry over: spilling is impossible because states cannot be copied, and freeing requires uncomputation or measurement.

D. Identified research gaps

Table I summarises the mechanisms. Three gaps motivate this work:

G1 Uncomputation-based reuse [2], [8]–[10] and measurement-based reuse [6], [7], [11] are developed as separate passes at different abstraction levels.

G2 Reclamation legality is stated as type rules [1], [3] or circuit-graph conditions [8] rather than as data-flow facts that compose with the liveness, scheduling and allocation passes of a conventional compiler.

G3 Static guarantees that a freed qubit is clean, and diagnostics for programs where no safe reclamation exists, are tied to specific languages, and their precision is rarely measured.

TABLE I
QUBIT-RECLAMATION MECHANISMS IN RELATED WORK

Work	Mechanism	Decision basis
Silq [1], Qurts [3]	implicit uncomputation	type system; lexical scope / lifetime
Qrisp [2]	uncomputation, recycling	Unqomp-style synthesis; workspace knob
Unqomp [8], Reqomp [9]	uncomputation (+ recompute)	circuit graph; ancilla budget
SQUARE [10]	uncompute or keep	cost model at call sites
CaQR [6], DeCross [7]	measure + reset	circuit DAG; exact / greedy
Jiang [11], Classiq [12]	reordering for reuse	dependency graph; topological sort
QRECLAIM	uncompute, measure + reset, DGE	data-flow window (Eq. 7), static validation, ERF budget

III. Proposed Model / Architecture

A. System overview

Fig. 1 shows the pipeline. Programs written in an embedded front end become QR-IR, a straight-line list of instructions over *logical* qubits. A linear type check rejects aliasing. A forward basis-state analysis and a backward liveness analysis then drive the *level-1 reclamation pass*, which inserts uncomputations, early measurements, resets and `free`s for programmer ancillas, and a static validator proves every inserted `free` safe. Multi-controlled gates are then lowered to Toffoli chains, and the same pass and validator reclaim the helper qubits this creates (level 2). Finally, a pressure-aware scheduler reorders commuting instructions, an ERF linear-scan allocator maps logical to physical qubits under an optional budget, and code is emitted as a Qiskit circuit or OpenQASM 2.

B. Intermediate representation (QR-IR)

A QR-IR program is a sequence $P = \iota_0 \dots \iota_{N-1}$ of

$$\iota ::= \texttt{alloc}\ q \mid \texttt{free}\ q \mid g^{\theta}(\bar{c}^{\bar{\pi}}; \bar{t}) \mid \texttt{measure}\ q \to c \mid \texttt{reset}\ q$$

where $\bar{c}$ are controls with polarities $\pi \in \{+, -\}$ (a $-$ control fires on $|0\rangle$), $\bar{t}$ are targets and θ optional angles. CX, Toffoli and k-controlled X are the same base operation `x` with $|\bar{c}| = 1, 2, k$. Qubits are *data* (owned by the programmer), *ancillas* (reclaimed by the compiler) or *helpers* (created by lowering). Gates fall into three classes by their action on computational-basis states: $G_{\mathrm{qf}} = \{\texttt{x}, \texttt{swap}\}$ *permute* the basis; $G_{\mathrm{dg}} = \{\texttt{z}, \texttt{s}, \texttt{t}, \texttt{rz}, \texttt{p}\}$ and their inverses are *diagonal*; $G_{\mathrm{gen}} = \{\texttt{h}, \texttt{rx}, \texttt{ry}\}$ may create superposition. The *write set* $W(\iota)$ contains the targets of $G_{\mathrm{qf}} \cup G_{\mathrm{gen}}$ gates and of `reset`; controls and diagonal gates never write. Negative controls stay symbolic until code generation, so their X conjugation is never mistaken for a write. QR-IR has no classically controlled

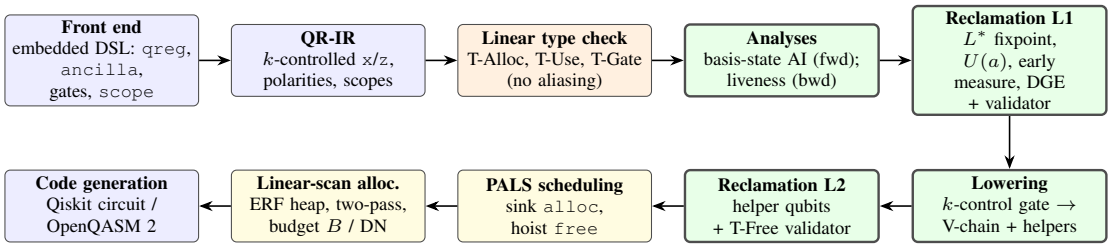


Fig. 1. The QRECLAIM pipeline. Stages with a bold border form the compile-time garbage collector; the same reclamation pass and validator run at two IR levels. A physical no-aliasing check and dead-reset elimination follow allocation.

operations, so after inlining and unrolling a program is a single basic block. Programmers write only the forward computation and never uncompute or free (Fig. 2).

C. Static semantics

Judgements $\Delta \vdash \iota \dashv \Delta'$ thread a context mapping each live qubit to its abstract state (§III-D1):

$$\frac{q \notin \mathrm{dom}(\Delta)}{\Delta \vdash \texttt{alloc}\ q \dashv \Delta, q{:}\mathbf{0}}\ \text{(T-ALLOC)}$$

$$\frac{q \in \mathrm{dom}(\Delta)}{\Delta \vdash \texttt{measure}\ q \dashv \Delta[q \mapsto \mathbf{M}]}\ \text{(T-USE)}$$

$$\frac{\bar{c}, \bar{t}\ \text{pairwise distinct} \quad \{\bar{c}, \bar{t}\} \subseteq \mathrm{dom}(\Delta)}{\Delta \vdash g(\bar{c}; \bar{t}) \dashv \Delta[\bar{t} \mapsto \tau_g(\Delta, \bar{t})]}\ \text{(T-GATE)}$$

$$\frac{\vdash_{XOR} q\ \text{clean}}{\Delta \vdash \texttt{free}\ q \dashv \Delta \setminus q}\ \text{(T-FREE)}$$

T-USE also covers `reset`, which sets the state to $\mathbf{0}$. The state transformer τ_g is that of Table II: G_{qf} yields $\mathbf{B}$ (or $\mathbf{T}$), G_{gen} yields $\mathbf{T}$, and G_{dg} changes nothing. Since QR-IR has no copy instruction, no program can clone a state [4]. The rules guarantee the two properties a qubit manager needs instead: an operation never uses one qubit in two roles, and a qubit is freed, and later handed out again, only when it is provably $|0\rangle$. The premise of T-FREE is discharged by the validator of §III-F. After allocation, a physical variant of the check verifies that no physical qubit hosts two live logical qubits.

D. Data-flow analyses

1) Basis-state abstract interpretation (forward): Each qubit carries a state $\sigma(q)$ in the lattice $\bot \sqsubseteq \mathbf{0} \sqsubseteq \mathbf{B} \sqsubseteq \mathbf{T}$, $\bot \sqsubseteq \mathbf{M} \sqsubseteq \mathbf{T}$, where $\bot$ means unallocated, $\mathbf{0}$ exactly $|0\rangle$, $\mathbf{B}$ a basis value that is a reversible function of other qubits, $\mathbf{M}$ measured and $\mathbf{T}$ possibly in superposition. It also carries a *trace* $Tr(q)$ (the qfree gates that computed it), a *dependency set* $Dep(q)$ (their controls) and a *deadline* $D(q)$: the first later position that writes some $d \in Dep(q)$. With $\mathrm{IN}[0] = \bot^Q$, $\mathrm{OUT}[i] = f_{\iota_i}(\mathrm{IN}[i])$ and $\mathrm{IN}[i{+}1] = \mathrm{OUT}[i]$, one forward sweep computes all facts; Table II gives f_ι. Clearing the reverse map from a written qubit to its dependants keeps the deadline bookkeeping amortised $O(1)$ per instruction.

TABLE II
TRANSFER FUNCTIONS f_ι OF THE BASIS-STATE ANALYSIS (ι AT POSITION i)

Instruction ι	Effect
`alloc` q	$\sigma(q){\leftarrow}\mathbf{0}$; $Tr(q){\leftarrow}\epsilon$; $Dep(q){\leftarrow}\emptyset$; $D(q){\leftarrow}\infty$
$g(\bar{c}; t)$, $g \in G_{\mathrm{qf}}$, $\sigma(t) \in \{\mathbf{0}, \mathbf{B}\}$, $D(t) = \infty$	$\sigma(t){\leftarrow}\mathbf{B}$; $Tr(t){\leftarrow}Tr(t) \cdot \iota$; $Dep(t){\leftarrow}Dep(t) \cup \bar{c}$
other qfree gate; $g \in G_{\mathrm{gen}}$	$\sigma(t){\leftarrow}\mathbf{T}$ for every target t
$g \in G_{\mathrm{dg}}$	no change (adds phases only)
`measure` q	$\sigma(q){\leftarrow}\mathbf{M}$
`reset` q	$\sigma(q){\leftarrow}\mathbf{0}$; $Tr(q){\leftarrow}\epsilon$; $Dep(q){\leftarrow}\emptyset$; $D(q){\leftarrow}\infty$
any ι, each $w \in W(\iota)$	for all a with $w \in Dep(a)$, $D(a) = \infty$: $D(a){\leftarrow}i$

2) Liveness (backward): With $DEF(\iota) = \{q\}$ for `alloc` q and $USE(\iota)$ the operands of every other instruction:

$$LIVE_{in}(\iota) = USE(\iota) \cup \big(LIVE_{out}(\iota) \setminus DEF(\iota)\big), \quad (1)$$

$$LIVE_{out}(\iota_i) = LIVE_{in}(\iota_{i+1}), \quad (2)$$

$$L(q) = \max\{\, i : q \in USE(\iota_i) \,\}. \quad (3)$$

One backward sweep yields every last use $L(q)$.

3) Uncomputation-aware liveness and placement: Inserting the uncomputation $U(a)$ of ancilla a at position $P(a)$ *reads* every $d \in Dep(a)$ again, so the last use of a dependency can move later. For the set R of ancillas to uncompute, the effective last use is the least fixpoint of

$$L^*(q) = \max\big(L(q),\ \max\{P(a) : a \in R,\ q \in Dep(a)\}\big), \quad (4)$$

$$P_{\mathrm{eager}}(a) = L^*(a), \quad (5)$$

$$P_{\mathrm{lazy}}(a) = \min\big(end(scope(a)),\ D(a){-}1,\ \min\{P_{\mathrm{lazy}}(b) : b \in Dep(a) \cap R\}\big), \quad (6)$$

and a placement is *legal* iff

$$L^*(a) \ \le\ P(a) \ <\ D(a) \qquad \text{for every } a \in R. \quad (7)$$

Eq. (5) is QRECLAIM's policy; Eq. (6) models lexical reclamation. If (7) fails, the program is rejected with a diagnostic naming the instruction that overwrote the input. The fixpoint is solved round-robin: P only grows and is bounded by N, so it terminates within $h{+}1$ sweeps for a dependency DAG of height h. On all our benchmarks one productive sweep plus one confirming sweep sufficed.

Theorem 1 (Sound uncomputation): Let every $a \in R$ satisfy (7), with $\sigma(a) = \mathbf{B}$ and $U(a) = Tr(a)_k^{-1} \cdots Tr(a)_1^{-1}$, and let blocks inserted at the same point be ordered so that a block reading q precedes the block writing (freeing) q. Then at each `free` a the ancilla is $|0\rangle$ and unentangled. Moreover the final state of the other qubits does not depend on which legal $P(a)$ is chosen.

Proof sketch: Every b with $a \in Dep(b)$ satisfies $P(b) \leq L^*(a) \leq P(a)$, and a reset of some $d \in Dep(a)$ is placed after $P(a)$, so no instruction, original or inserted, writes $Dep(a)$ between the first trace gate and $U(a)$. Expand the state in the computational basis of $Dep(a)$. In each branch $|d\rangle$, every trace gate sees the same control values when applied and when reversed. Every other operation on a only reads it (as a control or through a diagonal gate), since a write would have made $\sigma(a) = \mathbf{T}$. So $U(a)$ maps $|d\rangle|f(d)\rangle \mapsto |d\rangle|0\rangle$, and by linearity the state factorises as $|\psi\rangle \otimes |0\rangle_a$. Within the window, other instructions touch $Dep(a)$ only as controls, through diagonal gates or by measurement. They and $U(a)$ are block-diagonal in that basis and act on disjoint qubits within each block, so they commute. ∎

Lemma 1 (Early measurement): If the final operation on q is `measure` at m and its previous use is at $\ell < m$, moving the measurement to just after $\max(\ell, \max\{P(b) : q \in Dep(b)\})$ and appending `reset` q; `free` q preserves the joint outcome distribution and the state of all other qubits, because measurement commutes with operations on disjoint qubits and with operations that use q only as a control.

E. Reclamation transformations

Algorithm 1 summarises the pass, which is parameterised by the qubit kinds it manages.

Dead-garbage elimination (DGE). An ancilla whose only uses are its own trace gates is never read, so its computation and its uncomputation would cancel. Both are deleted together with the allocation, as for the unused carry-out of an adder.

Uncomputation, measurement and diagnostics. For each $a \in R$ the pass inserts $U(a)$; `free` a after $\iota_{P(a)}$, applies Lemma 1 to every qubit whose final operation is a measurement, and orders blocks at a shared point so that readers come before writers (every block ends in a `free`, which counts as a write). An ancilla that ends in $\mathbf{T}$ and is never measured is reported statically instead of being leaked.

Hierarchical reclamation. A gate with $k \geq 3$ controls is lowered to a V-chain of $k-2$ helpers and one Toffoli or CCZ. Lowering emits only the compute half; running Algorithm 1 again on helpers produces the uncompute half. Level 1 must run *before* lowering: there, a clause ancilla computed by a 3-controlled gate depends only on its inputs, whereas after lowering it would also depend on a helper, keeping that helper alive.

F. Static validation of frees

Rather than trusting the pass, QRECLAIM validates its output (translation validation). For each `free` a, consider the operations on a since its last `alloc` or `reset`. They must be X-type writes $a \oplus= \bigwedge(\bar{c}^{\bar{\pi}})$, uses of a as a control, or diagonal gates. Every control pattern $(\bar{c}, \bar{\pi})$ must occur an even number of times, and no control of a pattern may be written between that pattern's first and last occurrence. Each term is then XORed in an even number of times with identical control values on every basis branch, so a is $|0\rangle$ and unentangled, which discharges T-FREE. The validator runs after both reclamation levels in every compilation.

Algorithm 1 RECLAIM$(P, K, mode)$

$(\sigma, Tr, Dep, D) \leftarrow$ BASISAI(P); $L \leftarrow$ LIVENESS(P)
$G \leftarrow \{a \in K : \sigma(a) \in \{\mathbf{0}, \mathbf{B}\} \wedge uses(a) = Tr(a)\}$; delete $Tr(a)$, `alloc` a for $a \in G$
if $\exists a \in K \setminus G : \sigma(a) = \mathbf{T}$ **then error** "unreclaimable ancilla"
$R \leftarrow \{a \in K \setminus G : \sigma(a) = \mathbf{B}\}$
if $mode =$ EAGER **then**
    repeat for $a \in R$: $P(a) \leftarrow \max(L(a), \max\{P(b) : a \in Dep(b)\})$
    until no $P(a)$ changed
else $P(a) \leftarrow$ Eq. (6) by memoised recursion
check Eq. (7) for all $a \in R$, **else error** naming $D(a)$
for all q whose final operation is `measure` at m **do**
    move ι_m after $\max(uses(q) \setminus \{m\} \cup \{P(b) : q \in Dep(b)\})$; append `reset` q; `free` q
insert $U(a)$; `free` a after $\iota_{P(a)}$ for all $a \in R$, readers before writers

G. Scheduling and allocation

Pressure-aware list scheduling (PALS). Gates on disjoint qubits commute, so any topological order of the per-qubit precedence DAG is equivalent to the input. PALS emits ready instructions in order of their effect on the number of live qubits: `free` (-1) before other operations $(0$, in source order) before `alloc` $(+1$, ordered by first use). This sinks allocations and hoists reclamation, the program-level counterpart of CaQR's reuse pairs [6] and of reordering for recycling [11], [12].

ERF linear scan. After scheduling, each logical qubit occupies one interval $[$`alloc`, `free`$]$, so interference forms an interval graph.

Proposition 1: For a fixed order, let $\omega = \max_i |A(i)|$, where $A(i)$ is the set of qubits allocated but not freed at position i. Linear scan that reuses a free physical qubit whenever one exists uses exactly ω qubits, the minimum [15]. A second pass with budget $B = \omega$ that takes fresh qubits first never fails.

When the allocator reuses a qubit, it takes the pooled qubit whose last operation finishes earliest in the ASAP schedule (earliest-ready-first). This minimises the false dependency the reuse creates, and a min-heap gives $O(\log n)$ selection, since a pooled qubit's ready time cannot change. With a budget B, fresh qubits are used until B exist. QRECLAIM-*min* uses $B = \omega$. QRECLAIM-*DN* (depth-neutral) scans B upwards for the smallest budget whose circuit is no deeper than the no-reuse circuit. A linear scan is used because depth is not monotone in B. Resets on qubits that are never handed out again are removed afterwards. With N instructions, analyses and checks are $O(N)$ and scheduling and allocation $O(N \log N)$.

```
x = p.qreg("x", 2); y = p.qreg("y", 2)
for q in x + y: p.h(q)
with p.scope():                          # oracle (-1)^{x.y}
    for i in range(2):
        t = p.ancilla("t")               # never freed by hand
        p.ccx(x[i], y[i], t); p.z(t)
for q in x + y: p.h(q); p.measure(q)
```

(b) SCOPE: 6 qubits

```
alloc t0            ; q4
ccx x0,y0 -> t0; z t0
alloc t1            ; q5
ccx x1,y1 -> t1; z t1
ccx x1,y1 -> t1     ; U(t1)
ccx x0,y0 -> t0     ; U(t0)
free t1; free t0
h ...; measure ...
```

(c) QRECLAIM: 5 qubits

```
alloc t0            ; q4
ccx x0,y0 -> t0; z t0
ccx x0,y0 -> t0     ; U(t0)
free t0             ; q4 freed
h x0; measure x0    ; hoisted
h y0; measure y0    ; hoisted
alloc t1            ; q4 reused
ccx x1,y1 -> t1; z t1; U(t1)
```

Fig. 2. Worked example (condensed compiler output). (a) Source with two implicit temporaries. (b) Lexical reclamation keeps both until scope end. (c) QRECLAIM reclaims t_0 at its last use and reuses its qubit for t_1.

TABLE III
BENCHMARK FAMILIES (29 INSTANCES)

ID	Program	Sizes	Stresses
BV	Bernstein–Vazirani, n-bit secret	n=8..128	measured data
HS	hidden shift (inner-product bent function), one temporary per product	n=4..64	flat temporaries
RC	repetition-code memory, r=d rounds, fresh syndrome qubit per check	d=3..11	mid-circuit measurement
ADD	ripple-carry adder, carries in ancillas, unused carry-out	n=4..64	dependency chain, DGE
GRV	Grover on planted 3-SAT, m=$2n$ clauses, one ancilla per clause	n=4..10	iterated oracle, lowering
QAOA	one MaxCut-QAOA layer on an n-cycle, edge phase via t=$x_i \wedge x_{i+1}$	n=4..64	shared inputs

IV. IMPLEMENTATION / EXPERIMENTAL SETUP

Toolchain. QRECLAIM is about 1,500 lines of Python 3.13, one module per stage of Fig. 1. We use a custom pass pipeline rather than LLVM or GCC because reclamation needs linear qubit values, qubit kinds and uncomputation, none of which LLVM IR models. Code generation targets Qiskit 2.5.2 [13], which also lowers every output to $\{\mathrm{CX}, U\}$ at optimisation level 1 so that all configurations are measured on a common footing. Correctness is checked with an exact simulator (NumPy) that branches on mid-circuit measurements and asserts at every `alloc` and `free` that the qubit is $|0\rangle$, and cross-checked with Qiskit Aer 0.17.2. Experiments ran single-threaded on an Intel Xeon (2.10 GHz, Linux). One script regenerates every table, figure and number in this paper.

Benchmarks. Table III lists the six families, written in the high-level style the paper targets: temporaries are allocated freely and never uncomputed by hand. QAOA was added specifically because consecutive temporaries share an input, which prevents reordering from shortening their lifetimes.

Configurations. **NR**: correct uncomputation, no reuse. **SCOPE**: lazy placement (Eq. 6) with stack (LIFO) reuse, an idealised model of lexical reclamation. **SB** (strong baseline): lazy placement combined with prior techniques, i.e. CaQR-style early measurement and reset, PALS reordering and our ERF allocator; it isolates what eager placement and DGE add. **QRECLAIM-min** and **QRECLAIM-DN** are as in §III-G. As an external baseline we ran **Qrisp** 0.9.9 on the same programs, written with its automatic uncomputation, with and without `workspace=2`.

Metrics. Width (physical qubits, the memory footprint), CX count (code size), depth after lowering, mid-circuit resets, and compile time. Because a reset is much slower than a gate on current hardware, we also report an ASAP duration under an assumed latency model (1q gate $= 1$, CX $= 2$, measure and reset $= 10$). This is a stated assumption, not device data.

Bug injection. Seven mutation operators (Table VII) inject single faults into six small benchmarks, giving 237 mutants. Ground truth is exact: an ancilla is reclaimable if, at some insertion point after its last use, reversing its trace leaves it $|0\rangle$ for *every* basis input. Inputs are purified with reference qubits, so one simulation covers all inputs. A mutant is faulty if any ancilla is not reclaimable, and aliasing mutants are always faulty.

V. RESULTS & DISCUSSION

A. Correctness

We simulated exactly every output of width ≤ 20 whose mid-circuit measurements create at most 1,024 branches: 59 of the 145 compiled outputs of the six families (all five configurations), plus 35 of 36 programs from a separate set of small verification instances (in which GRV uses m<$2n$). All reproduce the analytically expected distribution, such as the BV secret, the hidden shift, the injected-error syndromes, $(a{+}b) \bmod 2^n$, the closed-form Grover distribution and an independent ancilla-free QAOA statevector, with total variation distance $\leq 1.3\times10^{-14}$ and all run-time $|0\rangle$ assertions holding. Qiskit Aer agrees within sampling noise (TVD ≤ 0.027 at 8,192 shots). Statically, the validator discharged T-FREE for all 1944 frees in the QRECLAIM-min outputs.

B. Memory footprint, code size and depth

Fig. 3 and Table IV report the results. Averaged over 29 instances, QRECLAIM-min uses 53.8% fewer qubits than NR (geometric-mean width ratio 0.323), 42.8% fewer than SCOPE and 8.3% fewer than the strong baseline. By mechanism:

- *BV, RC:* early measurement (plus PALS for BV) reduces width to 2 and $d{+}1$, independent of the number of rounds. SCOPE cannot reuse measured qubits, and SB, which includes early measurement, matches QRECLAIM here.
- *HS, GRV:* reuse across temporaries and oracle calls. SB matches QRECLAIM, because PALS can hoist lazily placed uncomputations whose inputs are not shared.
- *QAOA:* consecutive temporaries share an input, so a lazily placed uncomputation cannot be hoisted past the next computation. Only eager placement (Eq. 5) reclaims them: $n{+}1$ qubits against $2n$ for NR, SCOPE and SB, with *identical* depth.
- *ADD:* carries form a chain ($L^*(c_i) \geq P(c_{i+1})$), so placement cannot help. DGE removes the unused carry-out, saving one qubit and 22 CX gates at n=64. The deadline analysis (Eq. 7), not eager placement, forces the textbook interleaving of carry uncomputation and sum computation.

Reclamation adds no gates: every configuration must uncompute for correctness, so CX counts equal NR's except where DGE fires. The extra operations are resets, one per reuse of a measured qubit. Width is bought with depth: QRECLAIM-min is $2.55\times$ deeper than NR (geometric mean), because reuse serialises work.

C. Width–depth trade-off

Fig. 4 sweeps the budget B. Depth falls steeply and reaches the NR depth at 33% (BV), 51% (GRV), 59% (RC) and 75% (HS) of the NR width. QRECLAIM-DN selects this point automatically and saves 37.7% of the qubits on average with no depth increase on any instance (maximum depth ratio 1.0). Under the slow-reset latency model, the duration of QRECLAIM-DN ranges from 0.819 to 2.194 times that of NR. Where measured qubits are reused (BV, RC), the remaining resets cost time that unit depth does not show. On such hardware the budget should be chosen with the latency model, which the allocator supports directly.

D. Leave-one-out ablation

Table V removes one component at a time from the full pipeline. Early measurement is essential for BV and RC (2→33 and 8→49 qubits without it), PALS for BV (2→33), eager placement for QAOA (17→32) and DGE for ADD (47→48). The ERF policy and the two-pass allocation leave width unchanged but reduce depth; GRV, for instance, has depth 1094 with LIFO reuse and 943 without the second pass, against 825 for the full pipeline. On BV, HS, RC and GRV, removing eager placement changes nothing, because PALS hoists lazily placed uncomputations; on HS either one suffices. Eager placement matters exactly when temporaries share inputs, as in QAOA.

E. External baseline: Qrisp

Table VI compares QRECLAIM-min with Qrisp on the shared families, with all outputs verified by exact simulation. The comparison is sobering and informative. Qrisp reaches the *same* width on HS and QAOA, which confirms that eager, dependency-aware reclamation is what a mature framework also achieves. It is narrower on GRV and ADD and shallower everywhere. The reason is gate synthesis rather than reclamation: Qrisp implements multi-controlled gates without helper qubits and pairs compute/uncompute Toffolis as cheaper phase-tolerant variants, whereas QRECLAIM lowers to Barenco V-chains with helpers. Qrisp recycles a measured variable when the programmer deletes it, whereas QRECLAIM infers measurement-based reuse automatically (BV, RC), checks it statically and exposes it through one budget. Adopting helper-free and phase-tolerant synthesis in QRECLAIM's lowering stage is orthogonal to reclamation and is left as future work.

F. Static checks under bug injection

Table VII reports the bug-injection study. All 175 faulty mutants were rejected, so recall is 100.0%, and no accepted mutant failed a run-time $|0\rangle$ assertion. Precision is 94.6%: 10 correct mutants were rejected. Each false alarm writes a dependency that later code restores, or that a remaining trace gate never reads, which the first-write deadline $D(a)$ cannot see. Overwriting a control *after* the last use was accepted every time, because eager placement uncomputes before the write. The three diagnostics are:

```
E1 aliasing violation at [1] 'cx x0 -> x0': qubit x0 appears
     twice in one operation
E2 cannot uncompute ancilla t0: its control x0 is modified
    by 'h x0' at [5] while t0 is still needed at [6]
E3 ancilla t0 can never be reclaimed: 'h t0' at [2] puts t0
     in superposition, and it is not measured
```

G. Compilation time and threats to validity

All main instances compile in at most 42.4 ms, and the linear depth-neutral search takes at most 4375.6 ms. On inputs scaled to 26,594 instructions, a log–log fit of compile time against size has slope 1.03 (median 10.3 μs per instruction), within 0.5–6.5 times the time of Qiskit's own level-1 transpilation of the output. Threats to validity: depth assumes all-to-all connectivity; SCOPE and SB are our models of prior strategies, not re-implementations of specific tools (Qrisp is the one external tool); the benchmarks are small to medium and chosen to isolate mechanisms; exact simulation is limited to about 20 qubits, beyond which the static validator and the linear and physical checks carry the guarantee.

VI. Conclusion & Future Work

This case study treated qubit reclamation as compiler memory management. A linear IR with static semantics, a basis-state abstract interpretation and an uncomputation-aware liveness analysis decide where reclamation is legal. A unified pass inserts uncomputation, early measurement and resets, and its output is validated statically. An earliest-ready-first linear scan exposes the width–depth trade-off through one budget. QRECLAIM reduced width by 53.8% on average, and by 37.7% with no depth penalty. It beat a strong baseline built from prior techniques where temporaries share inputs, matched the Qrisp framework's width on two of four shared families, and its static checks found every injected fault. Four extensions are technically meaningful:

1) **Recomputation-aware placement.** Chains such as ADD's carries block early reclamation because $L^*(c_i) \geq P(c_{i+1})$. Adding optional recomputation edges to Eq. (4) turns placement into a min-cost reversible pebbling problem under budget B [9], solvable by ILP for small dependency DAGs and by a fixpoint heuristic otherwise.
2) **Restoration-aware deadlines.** The false alarms come from writes that later code undoes. Generalising $D(a)$ to a set of open windows, which reopen when the XOR-cancellation test of §III-F shows that a dependency has been restored, would accept these programs without losing soundness.
3) **Reclamation-aware synthesis.** Choosing between helper-based V-chains, helper-free decompositions and phase-tolerant Toffoli pairs inside the hierarchical pass,

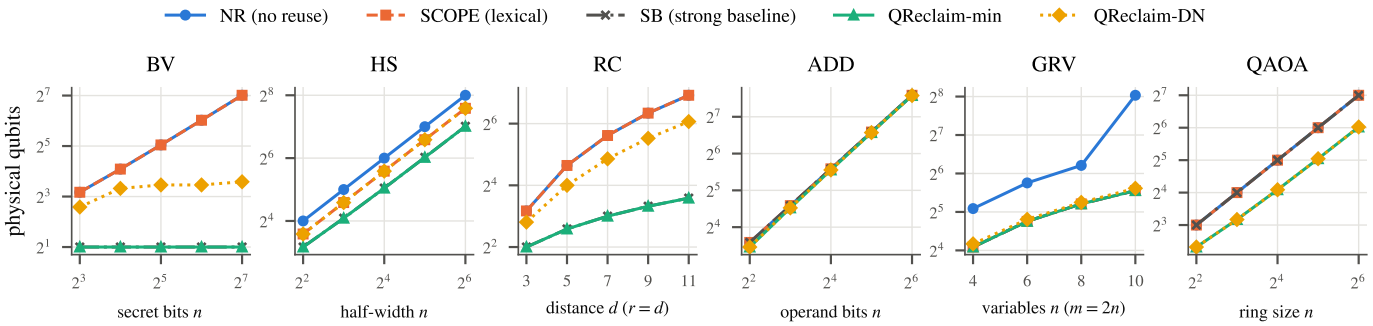


Fig. 3. Physical qubits (log scale) versus problem size. SB coincides with QRECLAIM-min except on ADD (DGE) and QAOA (eager placement); SCOPE coincides with NR on BV and RC.

TABLE IV
RESULTS AT THE LARGEST SIZE OF EACH FAMILY. WIDTH, DEPTH, CX, RESETS (QRECLAIM-MIN/QRECLAIM-DN) AND QRECLAIM COMPILE TIME

	Width (physical qubits)					Depth				CX		Resets	Compile
Benchmark	NR	SCOPE	SB	QRECLAIM-min	QRECLAIM-DN	NR	SB	QRECLAIM-min	QRECLAIM-DN	NR	QRECLAIM-min	min/DN	ms
BV-128	129	129	2	**2**	12	67	447	447	67	64	64	127/117	6.6
HS-64	256	192	129	**129**	192	44	1271	1271	44	1536	1536	31/0	15.3
RC-11	121	121	12	**12**	67	42	405	405	42	220	220	109/54	8.3
ADD-64	192	192	192	**191**	191	2962	2962	2935	2935	2267	2245	0/0	10.8
GRV-10	262	47	47	**47**	49	2455	2557	2557	2437	3060	3060	0/0	13.5
QAOA-64	128	128	128	**65**	65	1157	1157	1157	1157	768	768	31/31	6.0

TABLE V
LEAVE-ONE-OUT ABLATION: WIDTH/DEPTH (BV-32, HS-16, RC-7, ADD-16, GRV-8, QAOA-16)

Configuration	BV	HS	RC	ADD	GRV	QAOA
SCOPE	33/21	48/44	49/26	48/706	37/1094	32/293
SB	2/117	33/371	8/149	48/706	37/825	32/293
QRECLAIM (full)	2/117	33/371	8/149	47/679	37/825	17/293
−eager	2/117	33/371	8/149	47/679	37/825	32/293
−early-meas	33/21	33/581	49/26	47/679	37/825	17/293
−PALS	33/21	33/581	8/158	47/679	37/825	17/293
−DGE	2/117	33/371	8/149	48/706	37/825	17/293
−ERF (LIFO)	2/117	33/671	8/167	47/679	37/1094	17/377
−two-pass	2/117	33/371	8/158	47/679	37/943	17/293

TABLE VI
QRECLAIM-MIN VS. QRISP 0.9.9: WIDTH/DEPTH/CX AFTER THE SAME QISKIT LOWERING

Program	QRECLAIM-min	Qrisp	Qrisp (workspace 2)
HS-4	9/119/96	9/61/48	11/49/48
HS-8	17/209/192	17/84/96	19/61/96
HS-16	33/371/384	33/97/192	35/74/192
QAOA-4	5/77/48	5/39/24	7/25/24
QAOA-8	9/149/96	9/51/48	11/34/48
QAOA-16	17/293/192	17/72/96	19/53/96
GRV-4	17/478/372	12/266/166	14/194/148
GRV-6	27/663/588	18/377/258	20/315/250
ADD-4	11/115/85	10/56/41	12/56/41
ADD-8	23/303/229	22/136/105	24/136/105

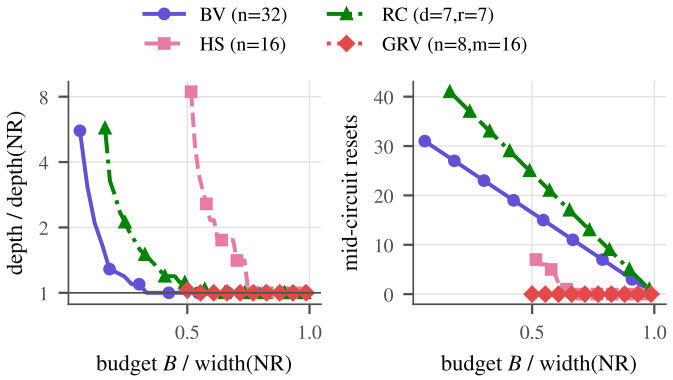


Fig. 4. Effect of the qubit budget B (normalised by the NR width). Left: depth relative to NR. Right: mid-circuit resets.

using the free pool and the budget, would combine Qrisp's synthesis gains with QRECLAIM's placement.

4) **Hardware-aware ERF and feed-forward.** Replace unit latencies with calibrated gate, measurement and reset times and add coupling-map distance to the ERF key. Lift the analyses from one basic block to control-flow graphs with classically controlled branches, which requires a join on the lattice and liveness across branches.

REFERENCES

[1] B. Bichsel, M. Baader, T. Gehr, and M. Vechev, "Silq: A high-level quantum language with safe uncomputation and intuitive semantics," in *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2020, pp. 286–300.

[2] R. Seidel, N. Tcholtchev, S. Bock, and M. Hauswirth, "Uncomputation in the Qrisp high-level quantum programming framework," in *Proc. Reversible Computation (RC), LNCS vol. 13960*. Springer, 2023, pp. 150–165.

[3] K. Hirata and C. Heunen, "Qurts: Automatic quantum uncomputation by affine types with lifetime," *Proc. ACM Program. Lang.*, vol. 9, no. POPL, pp. 155–182, 2025.

[4] W. K. Wootters and W. H. Zurek, "A single quantum cannot be cloned," *Nature*, vol. 299, no. 5886, pp. 802–803, 1982.

TABLE VII
BUG INJECTION: MUTANTS, FAULTY (GROUND TRUTH), TP, FN, FP, TN OF THE STATIC CHECKS

Operator	n	faulty	TP	FN	FP	TN
alias an operand	40	40	40	0	0	0
overwrite control (early)	40	31	31	0	3	6
overwrite control (late)	40	36	36	0	4	0
flip and restore control	20	17	17	0	3	0
reset a control	40	34	34	0	0	6
H on ancilla	17	17	17	0	0	0
overwrite after last use	40	0	0	0	0	40
total	237	175	175	0	10	52

[5] C. H. Bennett, "Logical reversibility of computation," *IBM J. Res. Develop.*, vol. 17, no. 6, pp. 525–532, 1973.

[6] F. Hua, Y. Jin, Y. Chen, S. Vittal, K. Krsulich, L. S. Bishop, J. Lapeyre, A. Javadi-Abhari, and E. Z. Zhang, "CaQR: A compiler-assisted approach for qubit reuse through dynamic circuit," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS), vol. 3*, 2023, pp. 59–71.

[7] M. DeCross, E. Chertkov, M. Kohagen, and M. Foss-Feig, "Qubit-reuse compilation with mid-circuit measurement and reset," *Phys. Rev. X*, vol. 13, no. 4, 2023, art. no. 041057.

[8] A. Paradis, B. Bichsel, S. Steffen, and M. Vechev, "Unqomp: Synthesizing uncomputation in quantum circuits," in *Proc. 42nd ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2021, pp. 222–236.

[9] A. Paradis, B. Bichsel, and M. Vechev, "Reqomp: Space-constrained uncomputation for quantum circuits," *Quantum*, vol. 8, 2024, art. no. 1258.

[10] Y. Ding, X.-C. Wu, A. Holmes, A. Wiseth, D. Franklin, M. Martonosi, and F. T. Chong, "SQUARE: Strategic quantum ancilla reuse for modular quantum programs via cost-effective uncomputation," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 570–583.

[11] H. Jiang, "Qubit recycling revisited," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, pp. 1264–1287, 2024.

[12] I. Reichental, R. Alon, L. Preminger, M. Vax, and A. Naveh, "Scalable memory recycling for large quantum programs," arXiv:2503.00822, 2025.

[13] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[14] N. Yu and J. Palsberg, "Quantum abstract interpretation," in *Proc. 42nd ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2021, pp. 542–558.

[15] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Trans. Program. Lang. Syst.*, vol. 21, no. 5, pp. 895–913, 1999.